

CÔNG TY CỔ PHẦN CÔNG NGHỆ OKAI



TÀI LIỆU HƯỚNG DẪN PHÁT TRIỂN
CH32V003F4P6

LỊCH SỬ UPDATE

Phiên bản	Ngày Update	Người Update	Ghi chú
V1.0	20/12/2025	Đào Trọng Khánh	Update lần đầu

Mục lục

LỊCH SỬ UPDATE.....	2
1. Giới thiệu về CH32V003F4P6.....	5
Các tính năng chính.....	5
2. Quy trình phát triển chip CH32V003F4P6.....	7
3. Hướng dẫn cài môi trường phát triển.....	8
3.1. Phát triển trên Vscode.....	8
3.1.1 Cài đặt.....	8
3.1.2. Tạo dự án.....	12
3.2. Phát triển trên PlatformIO.....	24
3.2.1 . Cài đặt nền tảng CH32V:.....	25
3.2.2. Tạo project.....	28
3.2.2.1. Tạo thủ công.....	28
3.2.2.2. Tạo từ example có sẵn.....	32
4. Một số lỗi thường gặp.....	32
4.1. Không connect được chip.....	32

Lời nói đầu

Tài liệu này được biên soạn bởi Công ty Cổ phần Công nghệ OKAI nhằm hướng dẫn quy trình phát triển và nạp chương trình cho VĐK CH32V003F4P6 của hãng WCH (Jiangsu Qinheng Microelectronics). Nội dung tài liệu tập trung vào các bước thực hành kỹ thuật cụ thể, giúp kỹ sư và kỹ thuật viên có thể thiết lập môi trường phát triển, tạo – biên dịch dự án và nạp chương trình (flash) cho CH32V003F4P6 một cách nhanh chóng và chính xác.

Tài liệu hướng dẫn chi tiết việc sử dụng VS Code và MounRiver Studio, xây dựng dự án với bộ biên dịch SDCC, cũng như cấu hình và sử dụng mạch nạp/debug WCH-LinkE để giao tiếp với vi điều khiển CH32V003F4P6 thông qua giao diện debug hai dây (SWD/RISC-V Debug).

Khác với các tài liệu giảng dạy lập trình hoặc đào tạo tư duy phần mềm, tài liệu này không đi sâu vào lý thuyết mà tập trung hoàn toàn vào thao tác kỹ thuật thực tế, nhằm giúp người dùng nhanh chóng làm chủ quy trình phát triển phần cứng và phần mềm cơ bản cho vi điều khiển CH32V003F4P6.

Bên cạnh các bước cài đặt và sử dụng công cụ, tài liệu còn tổng hợp các lỗi thường gặp trong quá trình biên dịch, kết nối debug và nạp chương trình, kèm theo phương án khắc phục cụ thể, được đúc kết từ kinh nghiệm triển khai thực tế trong các dự án nội bộ tại OKAI.

Với mục tiêu chuẩn hóa quy trình phát triển vi điều khiển CH32V003F4P6 trong các dự án của công ty, tài liệu này kỳ vọng sẽ trở thành nguồn tham khảo nội bộ đáng tin cậy, góp phần rút ngắn thời gian làm quen, đồng thời đảm bảo tính thống nhất và ổn định trong công tác phát triển và bảo trì sản phẩm.

1. Giới thiệu về CH32V003F4P6

CH32V003 là dòng vi điều khiển (MCU) đa năng cấp công nghiệp, được thiết kế dựa trên lõi QingKe RISC-V2A. Chip hỗ trợ tần số hệ thống lên đến 48MHz, nổi bật với các đặc tính: điện áp rộng, nạp chương trình/gỡ lỗi 1 dây (single-line debug), tiêu thụ điện năng thấp và kích thước gói siêu nhỏ.

CH32V003F4P6

1	PD4/A7/UCK/T2CH1ETR/OPO/T1CH4ETR_	PD3/A4/T2CH2/AETR/UCTS/T1CH4	20
2	PD5/A5/UTX/T2CH4_/URX_	PD2/A3/T1CH1/T2CH3_/T1CH2N	19
3	PD6/A6/URX/T2CH3_/UTX_	PD1/SWIO/AETR2/T1CH3N/SCL_/URX_	18
4	PD7/NRST/T2CH4/OPP1/UCK_	PC7/MISO/T1CH2_/T2CH2_/URTS	17
5	PA1/OSCI/A1/T1CH2/OPN0	PC6/MOSI/T1CH1CH3N_/UCTS_/SDA	16
6	PA2/OSCO/A0/T1CH2N/OPP0/AETR2_	PC5/SCK/T1ETR/T2CH1ETR_/SCL_/UCK_/T1CH3	15
7	VSS	PC4/A2/T1CH4/MCO/T1CH1CH2N	14
8	PD0/T1CH1N/OPN1/SDA_/UTX_	PC3/T1CH3/T1CH1N_/UCTS	13
9	VDD	PC2/SCL/URTS/T1BKIN/AETR_/T2CH2_/T1ETR	12
10	PC0/T2CH3/UTX_/NSS_/T1CH3_	PC1/SDA/NSS/T2CH4_/T2CH1ETR_/T1BKIN_/URX_	11

Các tính năng chính

- **Lõi (Core):**
 - Lõi QingKe 32-bit RISC-V, tập lệnh RV32EC.
 - Bộ điều khiển ngắt lập trình nhanh + ngăn xếp ngắt phần cứng.
 - Hỗ trợ ngắt lồng nhau 2 cấp.
 - Tần số hệ thống tối đa: 48MHz.
- **Bộ nhớ (Memory):**
 - 2KB SRAM (bộ nhớ dữ liệu tạm thời).
 - 16KB CodeFlash (bộ nhớ chương trình).
 - 1920B BootLoader.
 - 64B bộ nhớ cấu hình hệ thống không bay hơi.
 - 64B bộ nhớ do người dùng định nghĩa.

- **Quản lý năng lượng:**
 - Điện áp cung cấp (VDD): 3.3V hoặc 5V.
 - Chế độ năng lượng thấp: Sleep (Ngủ), Standby (Chờ).
- **Đồng hồ & Reset:**
 - Bộ dao động nội RC 24MHz (đã được tinh chỉnh tại nhà máy).
 - Bộ dao động nội RC 128KHz.
 - Bộ dao động ngoại tốc độ cao 4~25MHz.
 - Reset khi bật/tắt nguồn, bộ phát hiện điện áp có thể lập trình.
- **Ngoại vi & Giao tiếp:**
 - DMA: 1 bộ điều khiển DMA 7 kênh, hỗ trợ bộ đệm vòng (ring buffer).
 - ADC: 1 bộ ADC 10-bit (dải đo 0~VDD, 8 kênh ngoài + 2 kênh nội).
 - OPA/Comparator: 1 nhóm bộ khuếch đại thuật toán và bộ so sánh.
 - Timer (Bộ định thời):
 - 1 bộ timer 16-bit nâng cao (có kiểm soát dead-zone, phanh khẩn cấp, PWM cho điều khiển động cơ).
 - 1 bộ timer 16-bit đa năng.
 - 2 watchdog timer (độc lập và cửa sổ).
 - SysTick: bộ đếm 32-bit.
 - Giao tiếp: 1x USART, 1x I2C, 1x SPI.
- **Cổng I/O:**
 - 3 nhóm cổng GPIO, tổng cộng 18 cổng I/O.
 - Hỗ trợ ánh xạ 1 ngắt ngoài.
- **Tính năng khác:**
 - Bảo mật: ID duy nhất 64-bit (Unique ID).
 - Gỡ lỗi: Giao diện gỡ lỗi đơn dây (Serial single-wire debug - SDI).
 - Nhiệt độ hoạt động: -40°C đến 85°C (cấp công nghiệp).
 - Kiểu đóng gói: SOP, TSSOP hoặc QFN.

Datasheet chính thức của dòng vi điều khiển N76E003 tại trang của Nuvoton:

https://www.wch-ic.com/downloads/CH32V003DS0_PDF.html

2. Quy trình phát triển chip CH32V003F4P6

Quy trình phát triển chip CH32V003F4P6 có thể được chia thành 5 bước như sau:

Bước 1. Trình soạn thảo/môi trường phát triển tích hợp: Sử dụng các trình soạn thảo như Notepad++, Vscode,... .Hoặc các IDE như KEILC, MounRiver Studio(IDE của hãng) để viết chương trình cho CH32V003F4P6.

Bước 2. Trình biên dịch:

- **WCH RISC-V GCC**: Trình biên dịch chính thức do WCH cung cấp, được tích hợp sẵn trong **MounRiver Studio**, hỗ trợ đầy đủ thư viện ngoại vi và chức năng debug
- **GNU RISC-V GCC (mã nguồn mở)**: Trình biên dịch chuẩn cho kiến trúc RISC-V, miễn phí, phù hợp cho phát triển dự án độc lập, tự xây dựng Makefile và tích hợp CI.
- **SDCC (Small Device C Compiler)**: Trình biên dịch mã nguồn mở, miễn phí, thường được sử dụng cho các dự án nhỏ nhằm tạo mã gọn nhẹ và kiểm soát tốt dung lượng bộ nhớ, phù hợp với tài nguyên hạn chế của CH32V003F4P6.

Trong đó, **WCH RISC-V GCC** và **GNU RISC-V GCC** là lựa chọn phổ biến và ổn định nhất cho CH32V003F4P6, đặc biệt khi cần sử dụng đầy đủ các ngoại vi và công cụ debug. **SDCC** phù hợp cho các dự án đơn giản hoặc mục đích nghiên cứu, đào tạo nội bộ.

Bước 3. Tạo file hex: Các công cụ trình biên dịch sẽ tạo ra file này, Nếu dùng trình biên dịch (không cài công cụ mở rộng) phải gõ Terminal để tạo ra file hex hoặc tạo ra một makefile hoặc dùng python để viết lệnh tự động hóa cho quá trình này.

Bước 4. Nạp chip: Sử dụng phương thức ICP chúng ta cần sử dụng mạch nạp/debug chuyên dụng như WCH-LinkE.

Bước 5. Chạy chip/Debug: sau khi nạp chip xong cần test xem mạch đã chạy đúng với yêu cầu đặt ra chưa nếu có lỗi cần debug chương trình.

3. Hướng dẫn cài môi trường phát triển

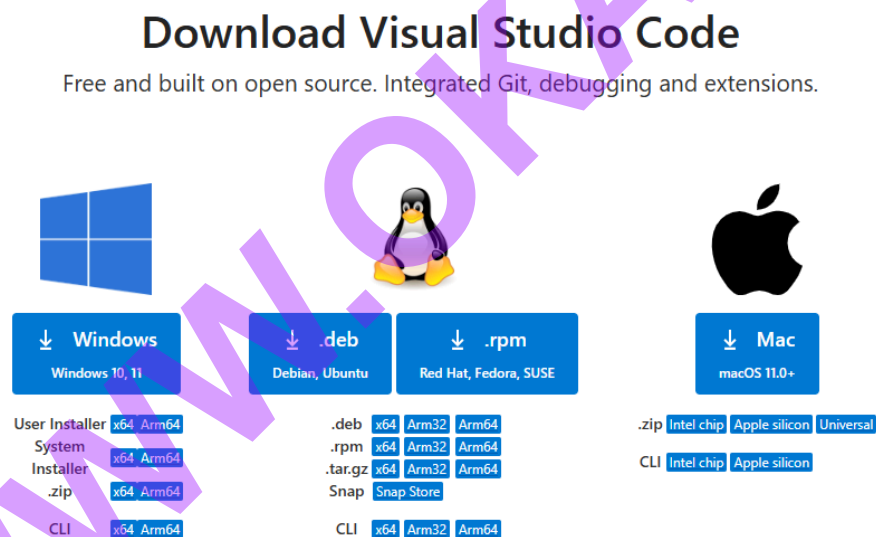
KEILC, IAR là những môi trường phát triển miễn phí có giới hạn, nếu muốn dùng toàn bộ tính năng cần có licence, ta sẽ dùng SDCC là trình biên dịch mở nguồn mở hoàn toàn miễn phí kết hợp với một trình soạn thảo.

3.1. Phát triển trên Vscode

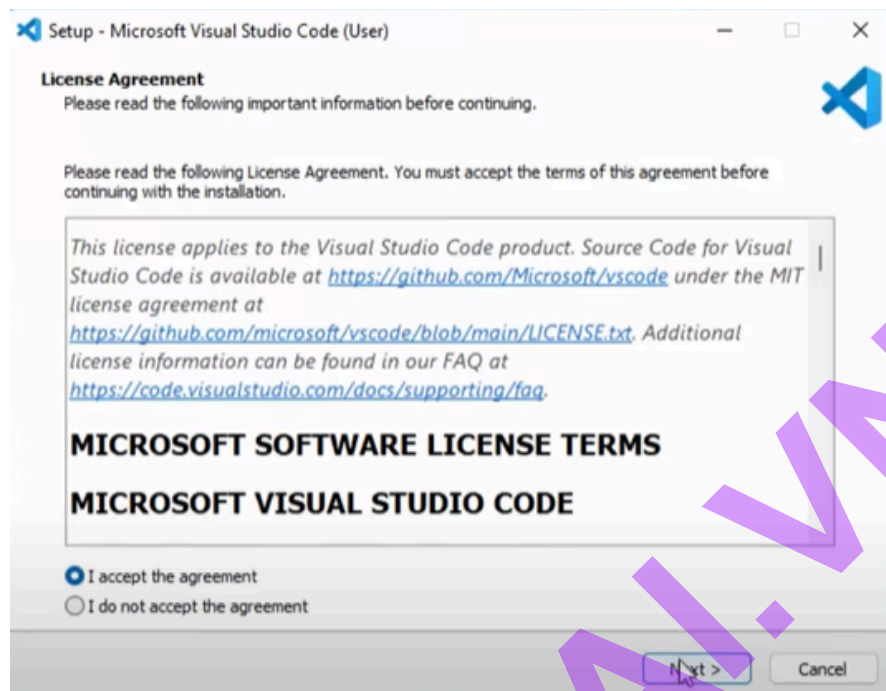
3.1.1 Cài đặt

Bước 1. Cài đặt VS code

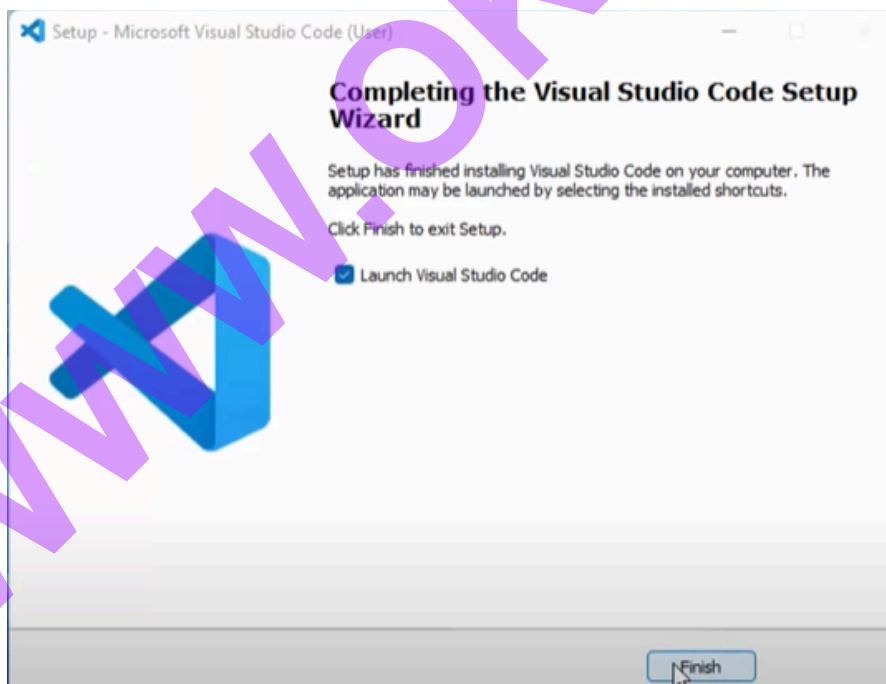
Tải VS code tại đường dẫn này: [download](#). Hãy lựa chọn đúng hệ điều hành bạn đang sử dụng trên máy tính. Hướng dẫn này sử dụng hệ điều hành Windows



Mở ứng dụng vừa download về, chọn “I accept the agreement”

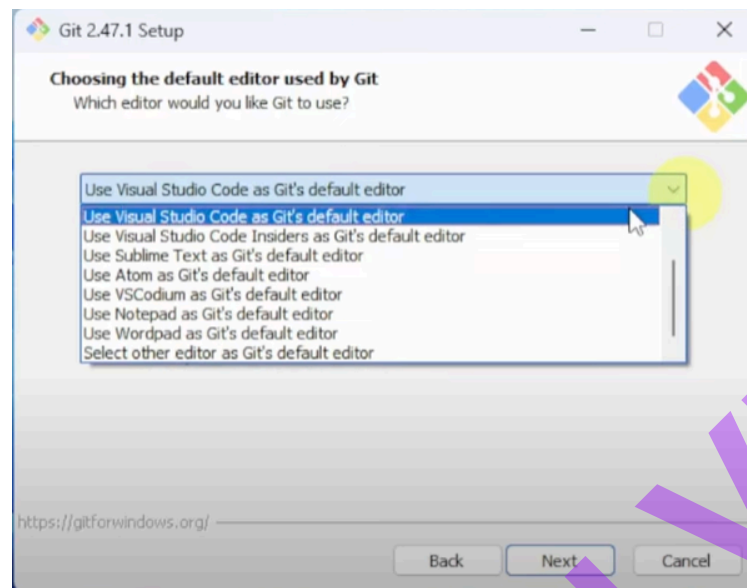


Ấn “Next” cho đến khi hiện chữ hiện chữ “Finish”

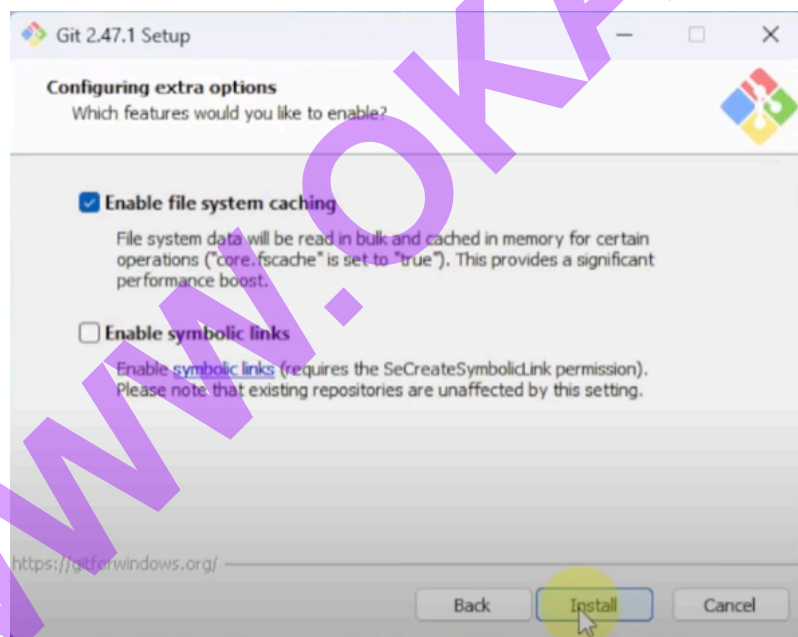


Bước 2. Cài đặt Git

Cài đặt git [tại đây](#). Click mở phần mềm ấn “Next” cho đến khi cửa sổ như hình bên dưới hiện lên chọn “Use Visual Studio Code as Git’s default editor”



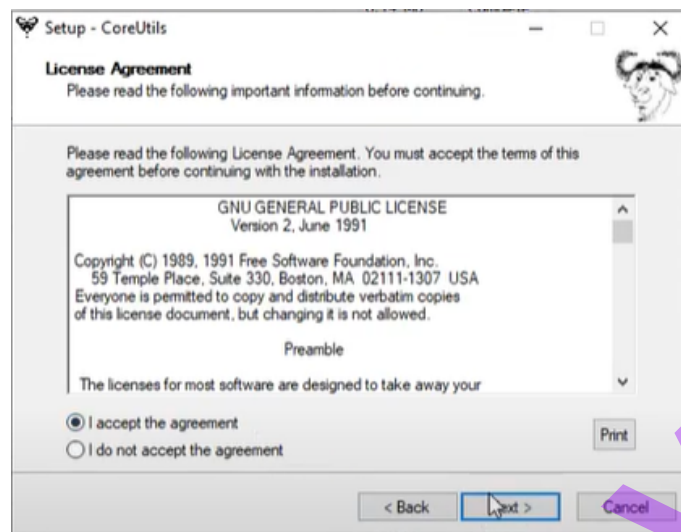
Ấn “Next” cho đến khi hiện chữ “Install”



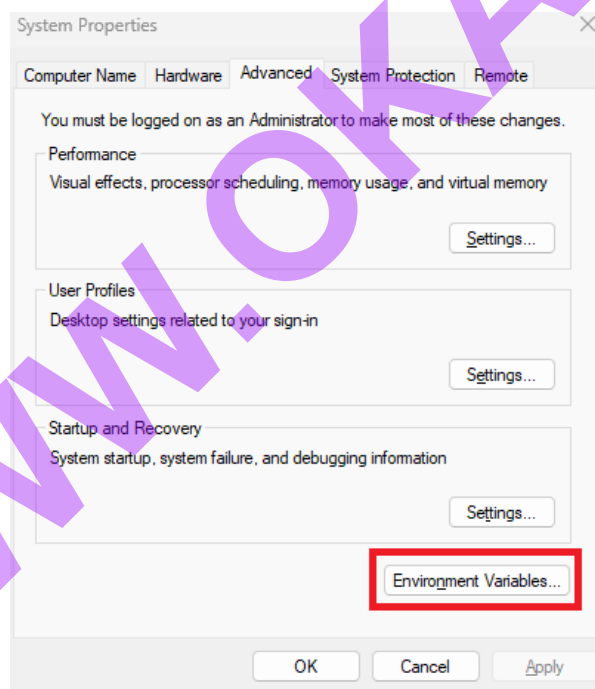
Ấn “Install” để cài đặt. Mở CMD nhập “git --version” để kiểm tra phiên bản.

Bước 3. Cài đặt Make, CoreUtils và SDCC

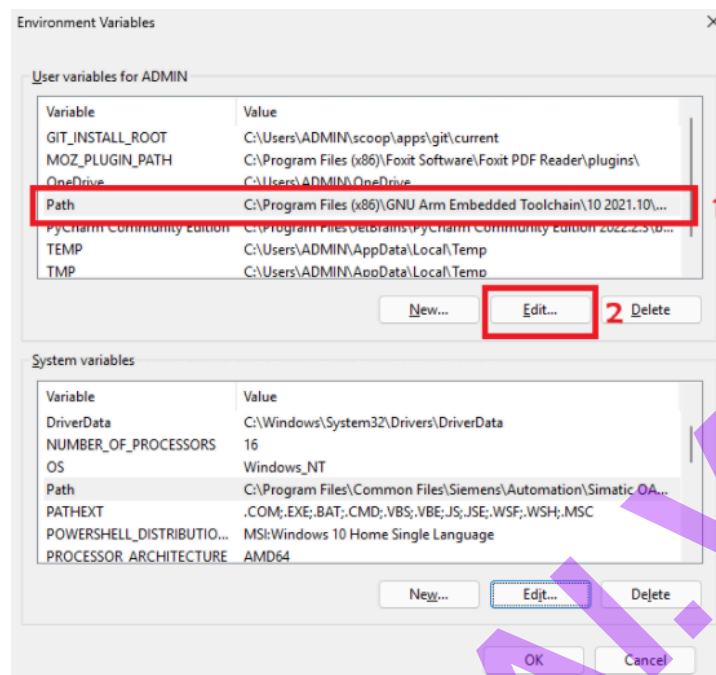
Cài đặt tại đây: [CoreUtils](#), [Make](#), [SDCC](#). Tương tự với những ứng dụng ở trên chọn “i accept the agreement” nếu có và ấn “Next” liên tục để cài đặt



Sau khi cài hết các ứng dụng trên ấn biểu tượng Window tìm từ khóa “edit the system environment variables” click chọn vào “Environment Variables...”



Tại cửa sổ “User variables for ADMIN” chọn “Path”->”Edit”



Ấn “New” sau đó điền đường dẫn “C:\Program Files (x86)\GnuWin32\bin” (điền đúng đường dẫn trên thiết bị của bạn). Để chắc chắn bạn có thể làm tương tự với mục System variable

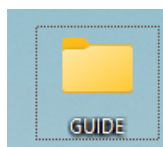
Bước 4. Cài đặt WCH-LinkUtility

Tải tại đây: [WCH-LinkUtility](#)

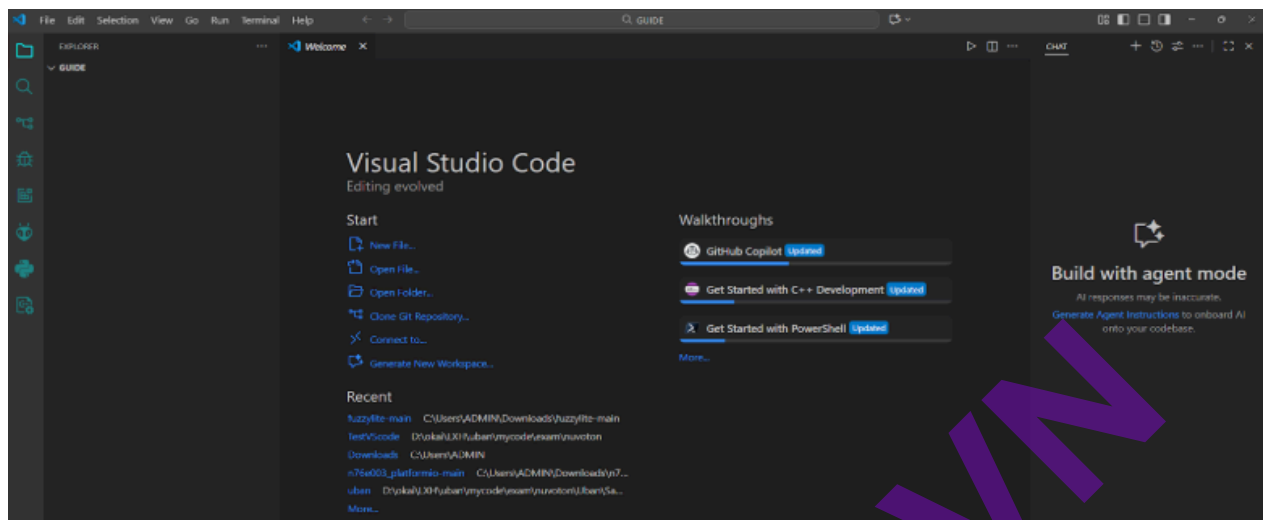
Cài đặt tương tự với các phần mềm ở trên

3.1.2. Tạo dự án

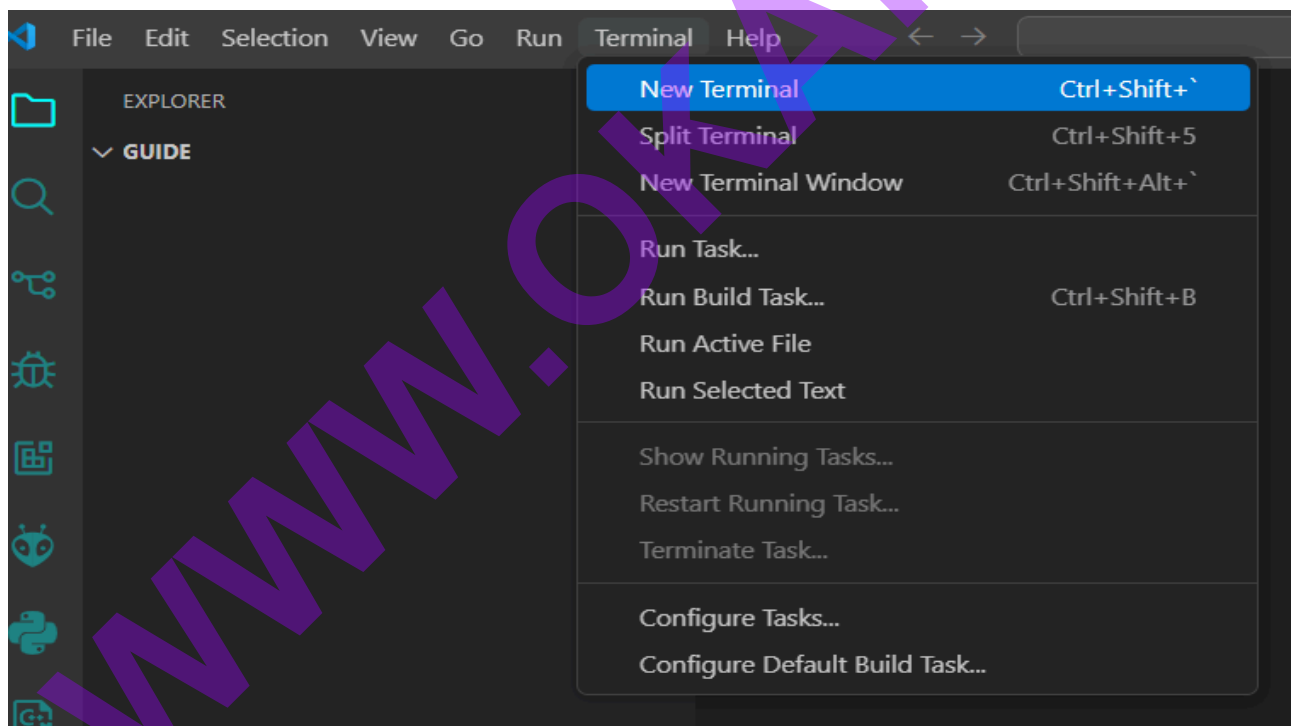
Tạo một thư mục trống ở đây tác giả đặt tên thư mục là “GUIDE”



Mở VS code, trên thanh công cụ ấn “File” -> “Open folder” -> tìm folder “GUIDE” vừa tạo -> “Select folder”



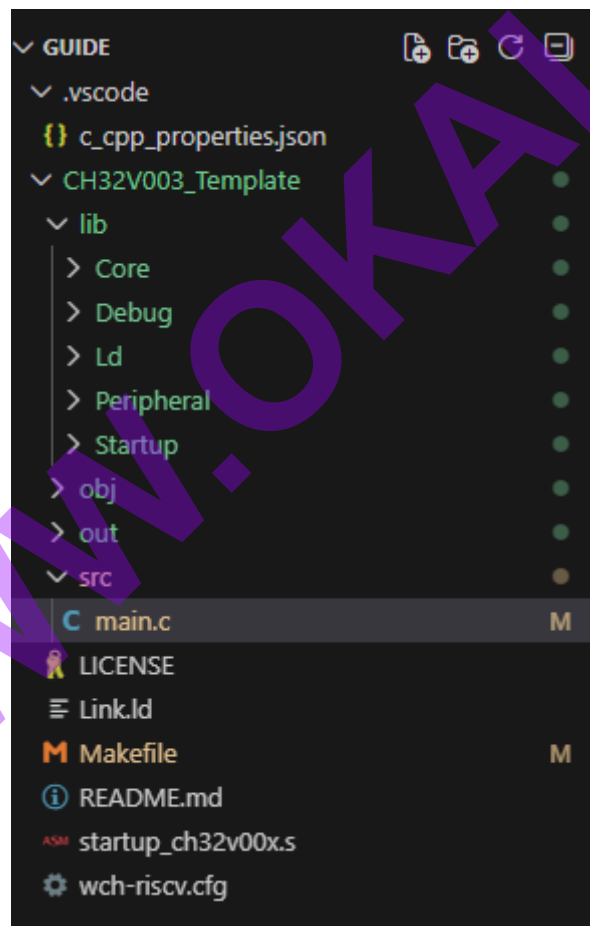
Trên thanh công cụ chọn “Terminal” -> “New Terminal”



Nhập “git clone https://github.com/TrKhanh02/CH32V003_Template.git” để tải thư viện mẫu.

```
PS C:\Users\Admin\Desktop\guide> git clone https://github.com/NgoHungCuong/CH32V003_Template.git
Cloning into 'CH32V003_Template'...
remote: Enumerating objects: 62, done.
remote: Counting objects: 100% (62/62), done.
remote: Compressing objects: 100% (51/51), done.
remote: Total 62 (delta 9), reused 58 (delta 9), pack-reused 0 (from 0)
Receiving objects: 100% (62/62), 110.36 KiB | 706.00 KiB/s, done.
Resolving deltas: 100% (9/9), done.
PS C:\Users\Admin\Desktop\guide> █
```

Sau khi tải xong góc trái màn hình sẽ hiển thị cấu trúc dự án như sau



lib: thư mục chứa file c của thư viện

obj: thư mục sau khi build thành sẽ chứa các asm, map, mem,...

out: mục chứa mã hex và bin để nạp vào chip

src: thư mục chứa chương trình chính main.c

Tải MounRiver Studio: <https://www.mounriver.com/download>

Cài đặt như bình thường

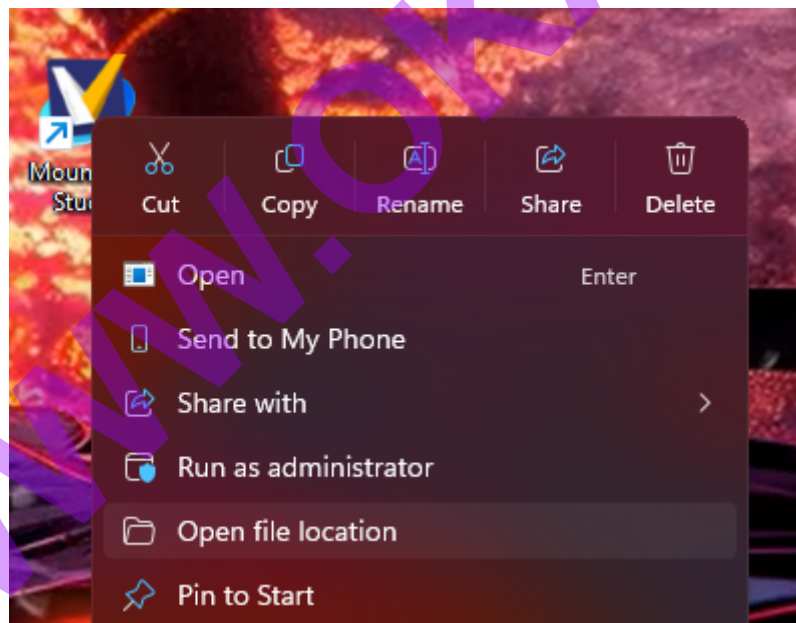
Nếu cài đặt mặc định theo phần mềm để thì không phải sửa gì trong makefile.

Nếu thay đổi đường dẫn thì hãy sửa đường dẫn GCC trong makefile như sau:

Tìm đoạn code này trong make file:

```
ifneq ($(OS),Windows_NT)
GCC_PATH = C:/MounRiver/MounRiver_Studio/toolchain/RISC-V Embedded GCC/bin
OCD_PATH = C:/MounRiver/MounRiver_Studio/toolchain/OpenOCD/bin
else
```

Sau đó tìm phần mềm MounRiver Studio: Click chuột phải Open file location.



Sau đó tìm theo đường dẫn sau : toolchain/RISC-V Embedded GCC/bin , sau đó copy đường dẫn đó paste vào GCC_PATH (lưu ý sửa “\” thành “/”)

OCD_PATH : làm tương tự

Trong main.c đã có một ví dụ như sau:

```
#include "debug.h"

int main(void)
{
    USART_Printf_Init(115200);

    GPIO_InitTypeDef gpioInit;

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);

    gpioInit.GPIO_Mode = GPIO_Mode_Out_PP;

    gpioInit.GPIO_Pin = GPIO_Pin_0;

    gpioInit.GPIO_Speed = GPIO_Speed_50MHz;

    GPIO_Init(GPIOC, &gpioInit);

    Delay_Init();

    while(1)
    {
        printf("LED Status: ON\r\n");

        GPIO_SetBits(GPIOC, GPIO_Pin_0);

        Delay_Ms(1000);

        GPIO_ResetBits(GPIOC, GPIO_Pin_0);

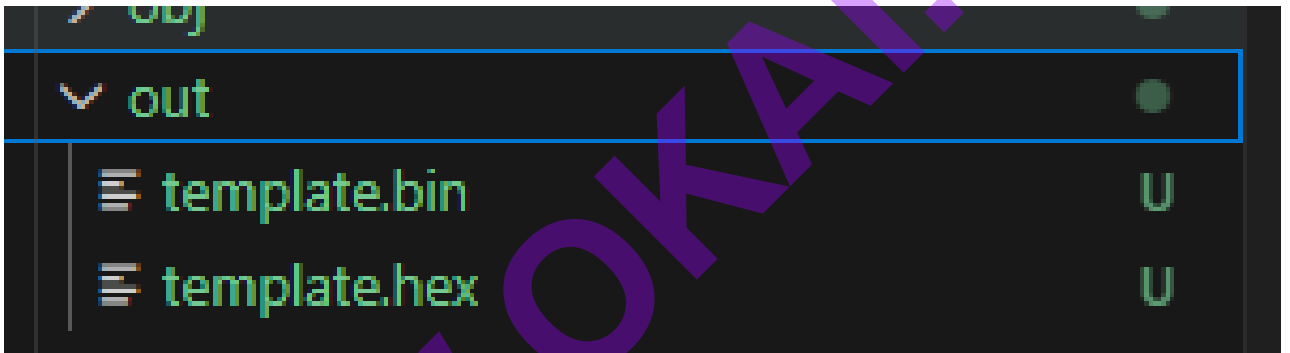
        printf("LED Status: OFF\r\n");

        Delay_Ms(1000);
    }
}
```

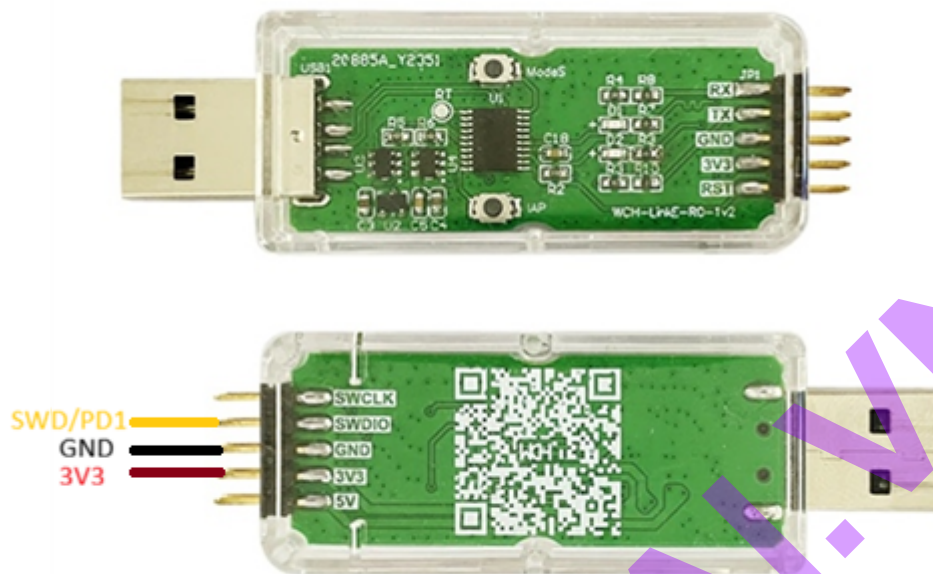
Ví dụ này là blink Led sau mỗi 1000ms

Để build ra file bin và hex, trên thanh công cụ chọn “Terminal”->”New Terminal” nhập lần lượt “cd CH32V003_Template” -> “Enter” -> “make clean”->”Enter”->”make”

```
PS G:\guide> cd CH32V003_Template
PS G:\guide\CH32V003_Template> make clean
rm -fr obj out
PS G:\guide\CH32V003_Template> make
mkdir -p obj
"C:/MounRiver/MounRiver_Studio/toolchain/RISC-V Embedded GCC/bin/riscv-none-embed-gcc" -c -march=rv32ecxw -mabi=ilp32e -msmall-data-limit=0 -msave-restore -fmessage-length=0 -fsigned-char -ffunction-sections -fddata-sections -fno-common -Wunused -Wuninitialized -Isrc -Ilib/Core -Ilib/Debug -Ilib/Peripheral/inc -Os -MD -MP -MF"obj/main.d" -std=gnu99 -MT"obj/main.o" "src/main.c" -o "obj/main.o"
"C:/MounRiver/MounRiver_Studio/toolchain/RISC-V Embedded GCC/bin/riscv-none-embed-gcc" -c -march=rv32ecxw -mabi=ilp32e -msmall-data-limit=0 -msave-restore -fmessage-length=0 -fsigned-char -ffunction-sections -fddata-sections -fno-common -Wunused -Wuninitialized -Isrc -Ilib/Core -Ilib/Debug -Ilib/Peripheral/inc -Os -MD -MP -MF"obj/system_ch32v00x.d" -std=gnu99 -MT"obj/system_ch32v00x.o" "lib/Core/system_ch32v00x.c" -o "obj/system_ch32v00x.o"
"C:/MounRiver/MounRiver_Studio/toolchain/RISC-V Embedded GCC/bin/riscv-none-embed-gcc" -c -march=rv32ecxw -mabi=ilp32e -msmall-data-limit=0 -msave-restore -fmessage-length=0 -fsigned-char -ffunction-sections -fddata-sections -fno-common -Wunused -Wuninitialized -Isrc -Ilib/Core -Ilib/Debug -Ilib/Peripheral/inc -Os -MD -MP -MF"obj/ch32v00x_it.d" -std=gnu99 -MT"obj/ch32v00x_it.o" "lib/Core/ch32v00x_it.c" -o "obj/ch32v00x_it.o"
"C:/MounRiver/MounRiver_Studio/toolchain/RISC-V Embedded GCC/bin/riscv-none-embed-gcc" -c -march=rv32ecxw -mabi=ilp32e -msmall-data-limit=0 -msave-restore -fmessage-length=0 -fsigned-char -ffunction-sections -fddata-sections -fno-common -Wunused -Wuninitialized -Isrc -Ilib/Core -Ilib/Debug -Ilib/Peripheral/inc -Os -MD -MP -MF"obj/debug.d" -std=gnu99 -MT"obj/debug.o" "lib/Debug/debug.c" -o "obj/debug.o"
```

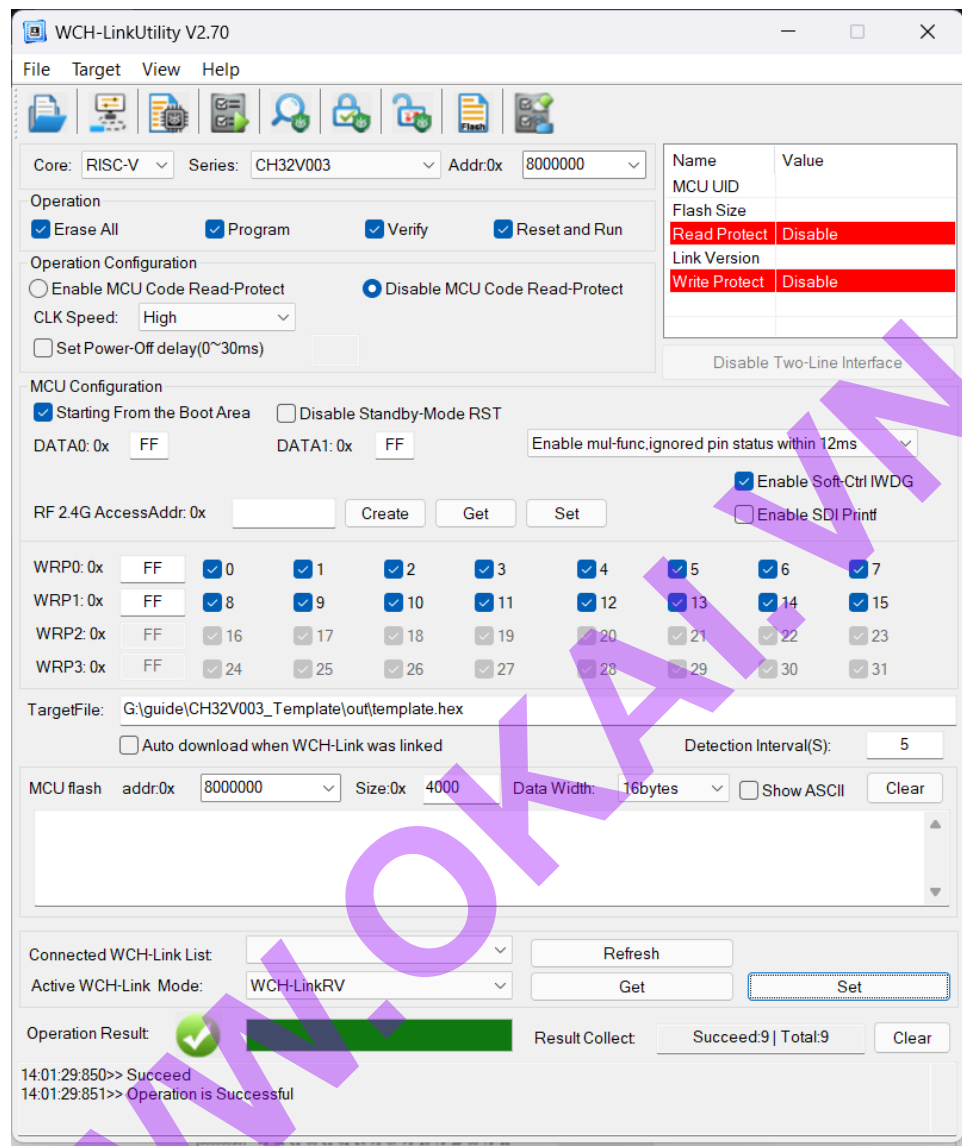


Cắm mạch nạp vào vào MCU theo bảng tín hiệu bên dưới



Mạch WCH-LinkE	CH32V003F4P6
3.3V	VCC
SWDIO	SWD/PD1
GND	GND

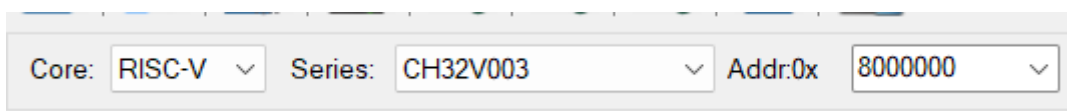
Sau đó mở phần mềm “ WCH Utility” tiến hành nạp chương trình.



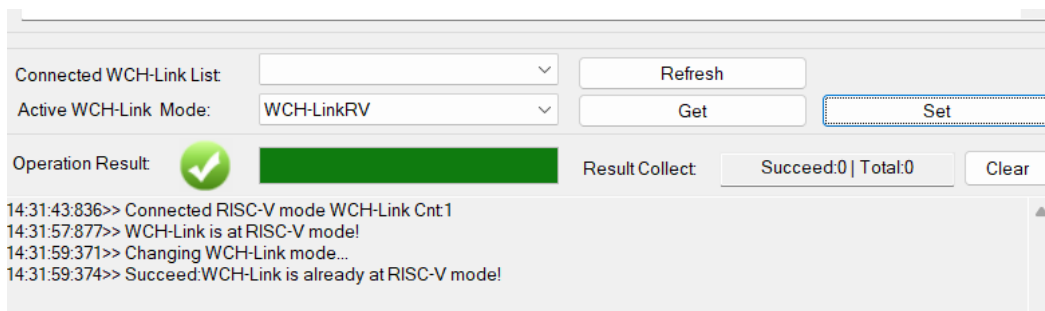
Trong này sẽ có 2 chế độ Core cho 2 loại chip:

- RISC-V: Đây là chế độ mình cần sử dụng
- ARM

Cắm mạch WCH-LinkE vào máy tính



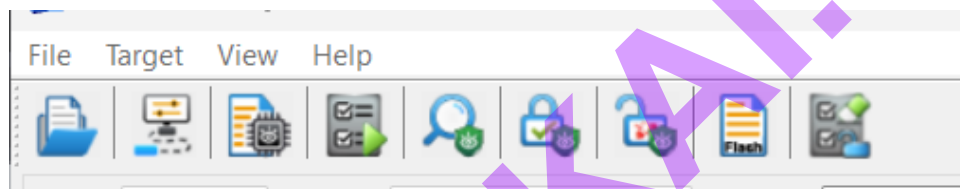
- Core chọn RISC-V, Series chọn CH32V003.



- Click lần lượt như sau để vào chế độ RISC-V/RV:

Refresh -> Active WCH-Link Mode phải hiển thị như hình -> Get -> Set

Thông báo Succeed:WCH-Link is already at RISC-V mode! là thành công



Thành công cụ: Chỉ cần chú ý thành công cụ sau.

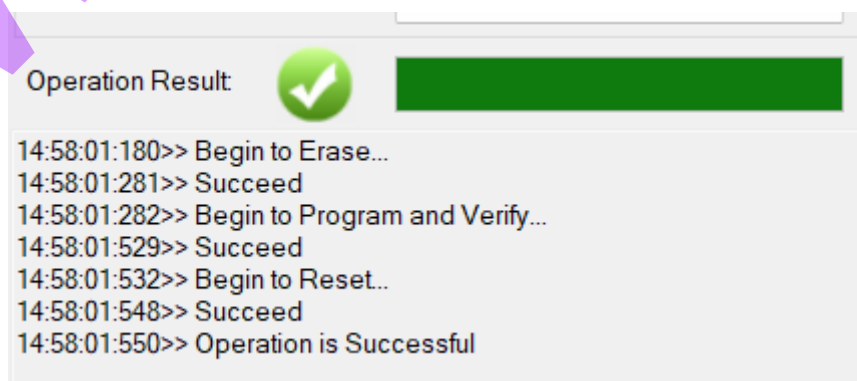
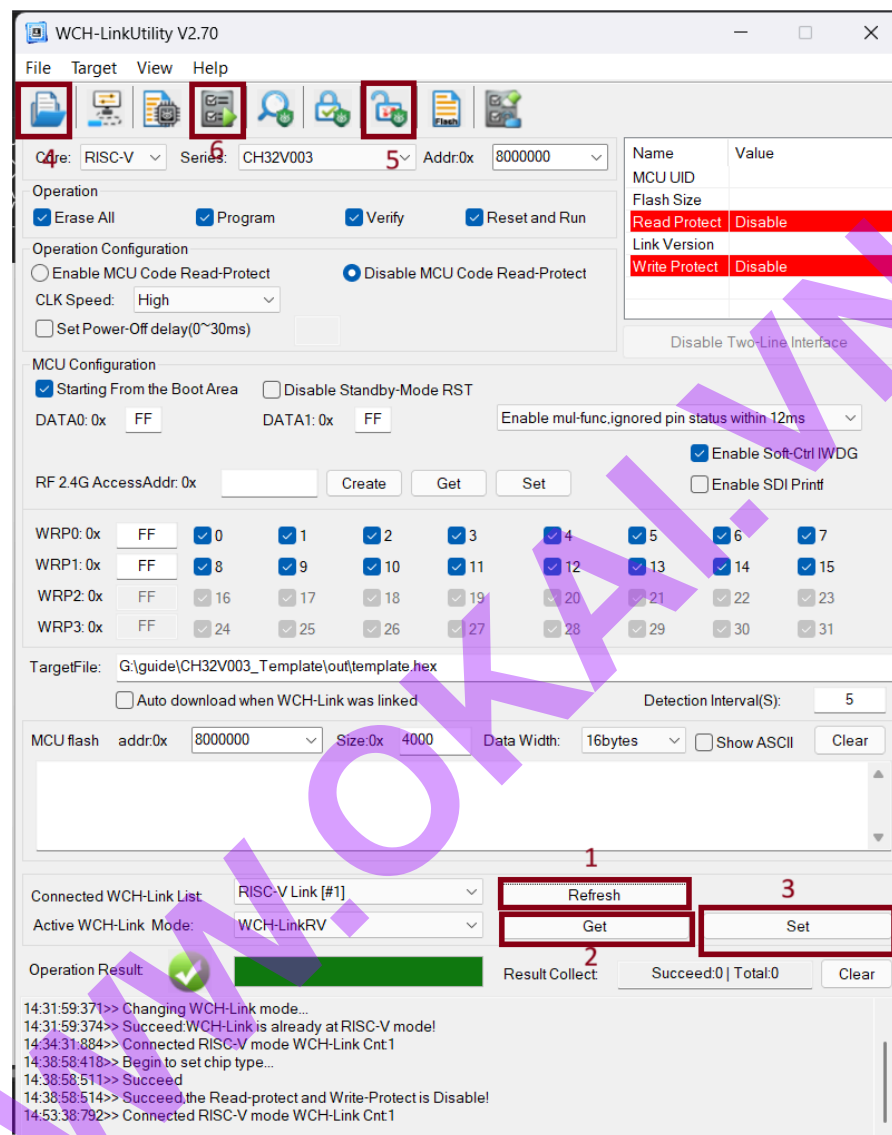
Chức năng:

1. Choose the file: chọn hex để nạp code (file hex đã build như trên).
2. **Disable Read-Protect** là lệnh dùng để gỡ bỏ lớp khóa bảo mật của chip (cần phải mở khóa để có thể nạp code được).
3. **Enable Read-Protect** (Bảo vệ đọc) là một cơ chế bảo mật phần cứng nhằm ngăn chặn việc sao chép hoặc can thiệp trái phép vào dữ liệu bên trong chip (hay còn được gọi là khóa chip không can thiệp vào chip).
4. **Clear Code Flash + Disable Protect + Execute**

Tác dụng: Đây là một tổ hợp lệnh tự động thực hiện 3 bước: Xóa sạch bộ nhớ Flash cũ -> Tắt chế độ bảo vệ (nếu có) -> Thực hiện nạp code mới.

Khi nào dùng: Bạn dùng nút này khi chip bị khóa hoặc khi bạn muốn đảm bảo chip hoàn toàn trống sạch trước khi nạp bản code mới nhất.

5. Execute Checked Operation: Gọi đơn giản đây là nút nạp code



Nếu cửa sổ hiển thị “Operation is Successful” chip đã được nạp thành công. Bạn có thể dùng mạch [USB to TTL](#) để đọc kết quả. Đảm bảo đấu đúng sơ đồ như sau. PD6 [TX] -> TTL[RX] , PD5 [RX] <- TTL[TX]

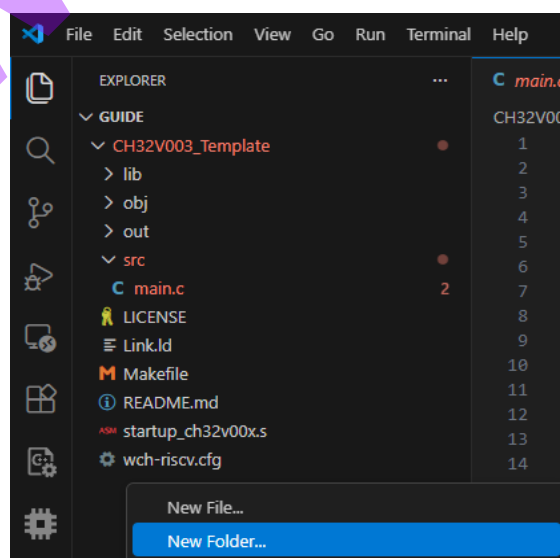
```
LED Status: OFF
LED Status: ON
LED Status: OFF
LED Status: ON
LED Status: OFF
LED Status: ON
LED Status: OFF
```

Trên đây là một trong “N” cách để phát triển CH32V003 trên vscode. Còn rất nhiều cách khác hay và thuận tiện cho lập trình viên phát triển MCU này, các tài liệu này sẽ được cập nhật sau.

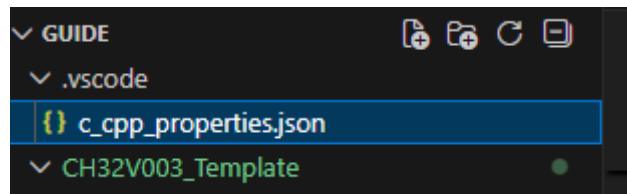
Fix gạch báo lỗi đỏ:

```
#include "debug.h"
#include errors detected. Please update your includePath. Squiggles are disabled for this translation
unit (C:\Users\Admin\Desktop\Guide\CH32V003_Template\src\main.c). C/C++(1696)
cannot open source file "stdint.h" (dependency of "debug.h"). Please run the 'Select IntelliSense
Configuration...' command to locate your system headers. C/C++(1696)
```

Bước 1: Tạo 1 folder mới ngang hàng với CH32V003_Template là .vscode



Bước 2: Trong thư mục .vscode tạo 1 file tên: **c_cpp_properties.json**



Bước 3: Paste đoạn code sau vào **c_cpp_properties.json**

```
{
  "configurations": [
    {
      "name": "Win32",
      "includePath": [
        "${workspaceFolder}/**",
        "${workspaceFolder}/CH32V003_Template/src",
        "${workspaceFolder}/CH32V003_Template/lib/Core",
        "${workspaceFolder}/CH32V003_Template/lib/Debug",
        "${workspaceFolder}/CH32V003_Template/lib/Peripheral/inc"
      ],
      "defines": [
        "_DEBUG",
        "UNICODE",
        "_UNICODE",
        "CH32V00x"
      ],
      "compilerPath": "C:/MounRiver/MounRiver_Studio/toolchain/RISC-V Embedded GCC/bin/riscv-none-embed-gcc.exe",
      "cStandard": "gnu99",
      "cppStandard": "gnu++14",
      "intelliSenseMode": "windows-gcc-arm"
    }
  ]
}
```

```

}

],

"version": 4
}

```

Sau đó sẽ thấy mất cảnh báo đỏ khi khai báo.

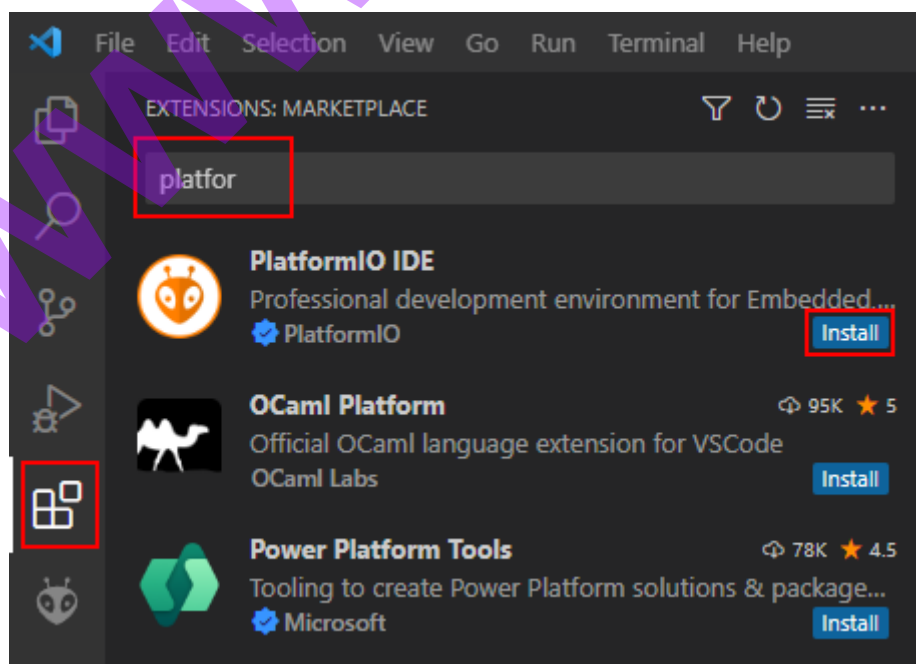
3.2. Phát triển trên PlatformIO

Vui lòng cập nhật phần mềm WCH-Link(E) bằng cách sử dụng

- [WCH-LinkUtiliti](#) (Chỉ dành cho Windows, Chọn Target -> Query Chip Info).
- [MounRiver Studio](#) (MounRiver Studio, giao diện đồ họa Eclipse cho Windows và Linux)

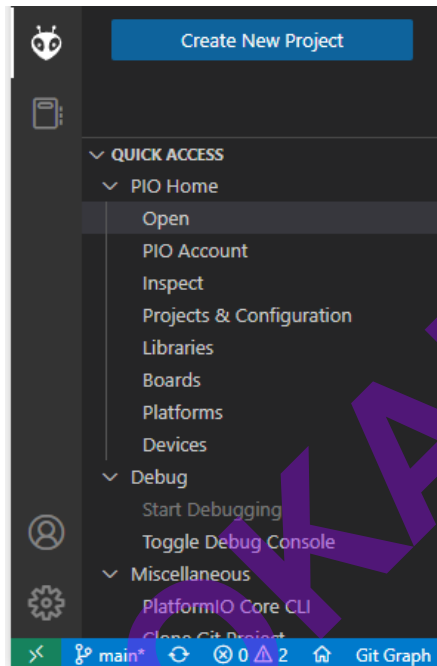
Install VsCode: Download and install VSCode from: <https://code.visualstudio.com/>.

Install PlatformIO: Open the “Extensions” sidebar, search for “PlatformIO” and hit “install”.

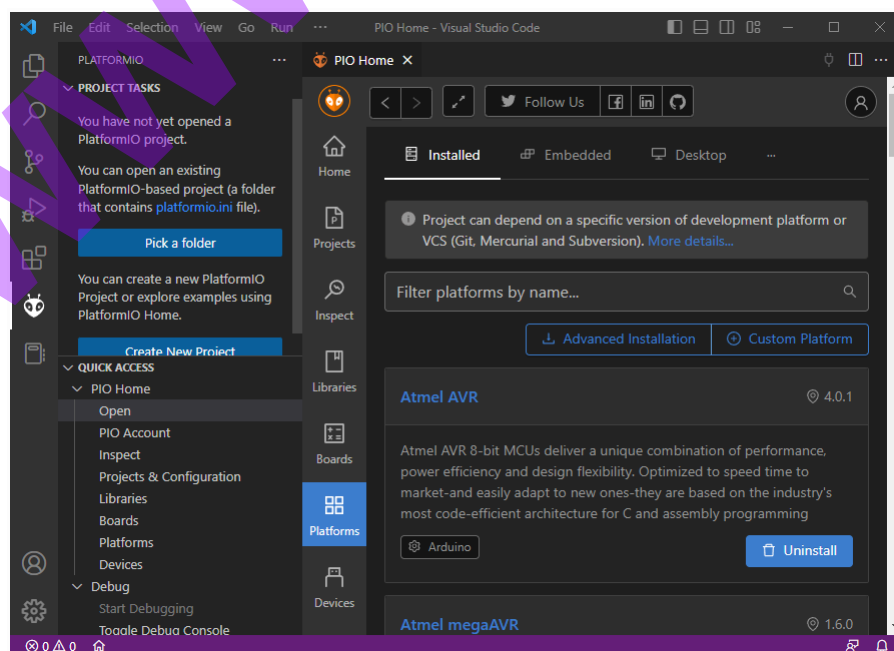


3.2.1 . Cài đặt nền tảng CH32V:

Cách 1: Mở rộng thanh bên PlatformIO (biểu tượng con kiến) và nhấp vào “Trang chủ PIO”.

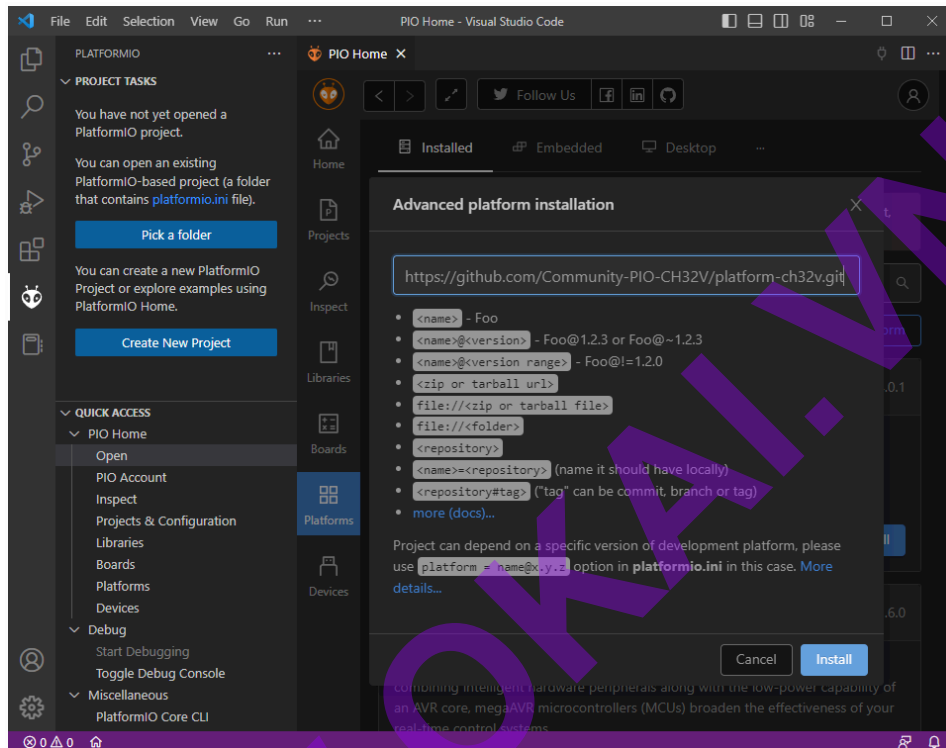


Trong cửa sổ PIO Home hiện ra, hãy nhấp vào thanh bên “Platforms” và chọn “Advanced Installation”.



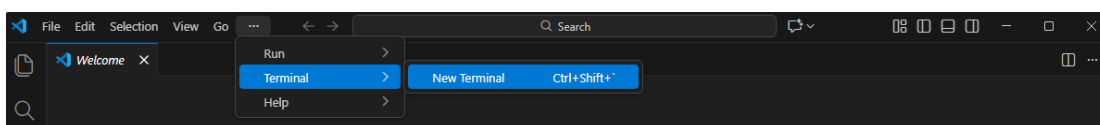
Bạn sẽ được yêu cầu cung cấp kho lưu trữ. Nhập URL và nhấn “Cài đặt”.

URL: <https://github.com/Community-PIO-CH32V/platform-ch32v.git>



Cách 2: Mở Vscode chỉ chuột vào thanh “...” -> Teminal -> New Teminal.

```
PS C:\Users\Admin> pio pkg install -g -p https://github.com/Community-PIO-CH32V/platform-ch32v.git
Platform Manager: ch32v@1.1.0+sha.100e1c1 is already installed
Tool Manager: Installing git+https://github.com/Community-PIO-CH32V/toolchain-riscv-windows.git
git version 2.51.1.windows.1
Cloning into 'C:\Users\Admin\.platformio\.cache\tmp\pkg-installing-dxq5gti2'...
remote: Enumerating objects: 2209, done.
remote: Counting objects: 100% (2209/2209), done.
remote: Compressing objects: 100% (1261/1261), done.
remote: Total 2209 (delta 901, reused 1421 (delta 868), pack-reused 0 (from 0))
Receiving objects: 100% (2209/2209), 126.85 MiB | 11.71 MiB/s, done.
Resolving deltas: 100% (901/901), done.
Updating files: 100% (2776/2776), done.
Tool Manager: toolchain-riscv@1.80200.190731+sha.8ee4117 has been installed!
Tool Manager: tool-openocd-riscv-wch@2.1100.240729 is already installed
Tool Manager: tool-wlink@0.23.241116+sha.f44158e is already installed
```



Sau đó nhập đoạn URL này vào Teminal mới tạo và Enter:

pio pkg install -g -p https://github.com/Community-PIO-CH32V/platform-ch32v.git

Như này là cài đặt môi trường thành công

Sau đó update: Nhập lệnh `pio pkg update -g -p "ch32v"`

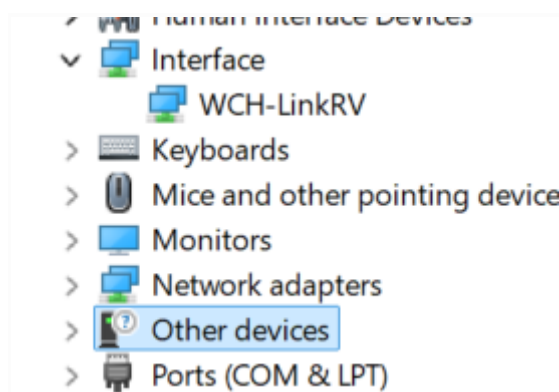
```
PS C:\Users\Admin\Desktop\CH32V003> pio pkg update -g -p "ch32v"
Platform Manager: ch32v@1.1.0+sha.100e1c1 is already up-to-date
Tool Manager: framework-arduino-openwch-ch32@0.0.0+sha.9cde8bf is already up-to-date
Tool Manager: framework-wch-noneos-sdk@2.30000.0+sha.f3f1eb7 is already up-to-date
Tool Manager: tool-minichlink@0.1.0+sha.af02ba5 is already up-to-date
Tool Manager: tool-openocd-riscv-wch@2.1100.240729 is already up-to-date
Tool Manager: tool-wchisp@0.23.240914 is already up-to-date
Tool Manager: tool-wlink@0.23.241116+sha.f44158e is already up-to-date
PS C:\Users\Admin\Desktop\CH32V003>
```

Cài đặt trình điều khiển Windows:

Việc nạp firmware cho các bo mạch phát triển thông qua đầu dò WCH-Link(E) (và kết nối SWCLK và/hoặc SWDIO) yêu cầu phải cài đặt trình điều khiển USB của W.CH cho thiết bị đó.

1. Tải xuống gói trình điều khiển WCHLink dành cho Windows.
2. Giải nén nó
3. Chạy chương trình WCHLink\SETUP.EXE và làm theo hướng dẫn cài đặt.
4. Chạy chương trình WCHLinkSER\SETUP.EXE và làm theo hướng dẫn cài đặt.

Nếu thành công, sau khi cắm thiết bị WCH-Link(E), bạn sẽ thấy một thiết bị thuộc loại “cổng nối tiếp” và “giao diện” trong trình quản lý thiết bị của Windows.

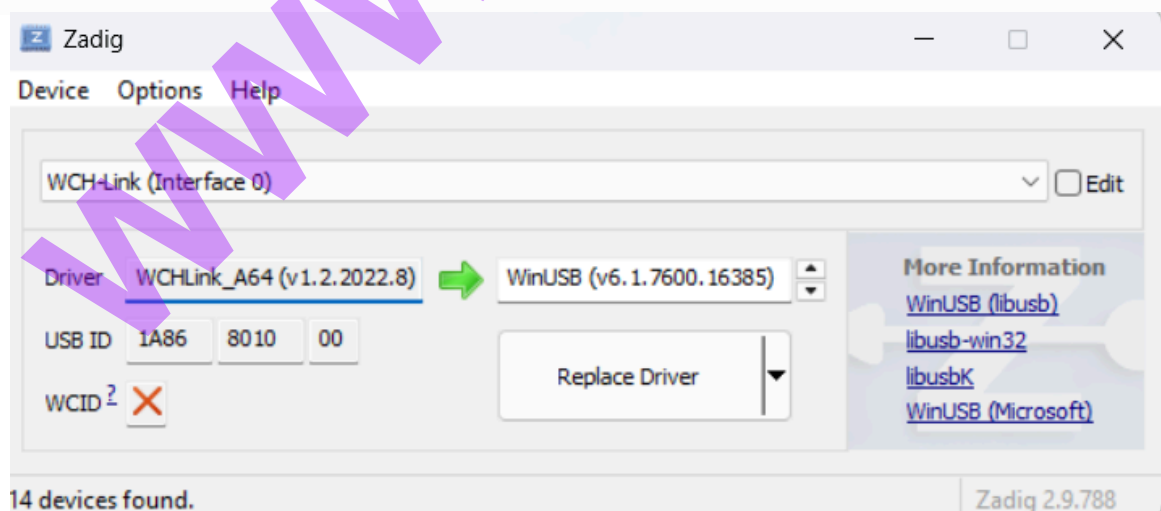


Việc nạp firmware cho các bo mạch phát triển thông qua bộ nạp khởi động USB tích hợp sẵn yêu cầu phải cài đặt trình điều khiển WinUSB cho thiết bị đó.

Nếu tải 2 driver trên đã có mục interface : WCH-Link như ảnh trên thì không cần tải Zadig nữa.

1. Tải xuống [Zadig](#)
2. Khởi động Zadig và kích hoạt Tùy chọn → Liệt kê tất cả thiết bị
3. Đưa bo mạch phát triển của bạn vào chế độ bootloader.
 - Đối với hầu hết các trường hợp: BOOT0 nối với 3.3V, BOOT1 nối với GND.
 - hoặc giữ nút có chữ “BOOT” được đánh dấu
4. Cắm bo mạch phát triển vào máy tính.
5. Chọn thiết bị “USB Module” trong Zadig.
6. Chọn “WinUSB” ở phía bên phải.
7. Nhấp vào “Thay thế trình điều khiển”

Giờ nó sẽ trông như thế này:

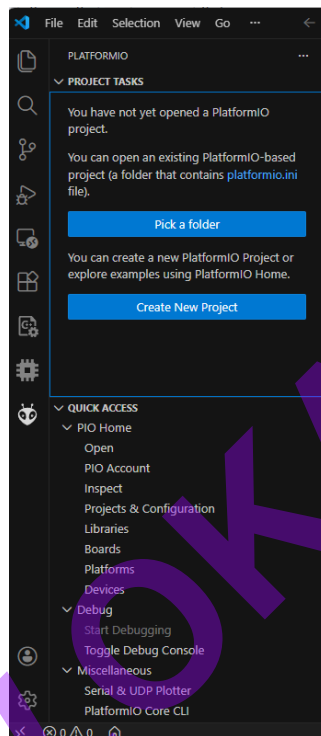


Như vậy đã cài xong môi trường:

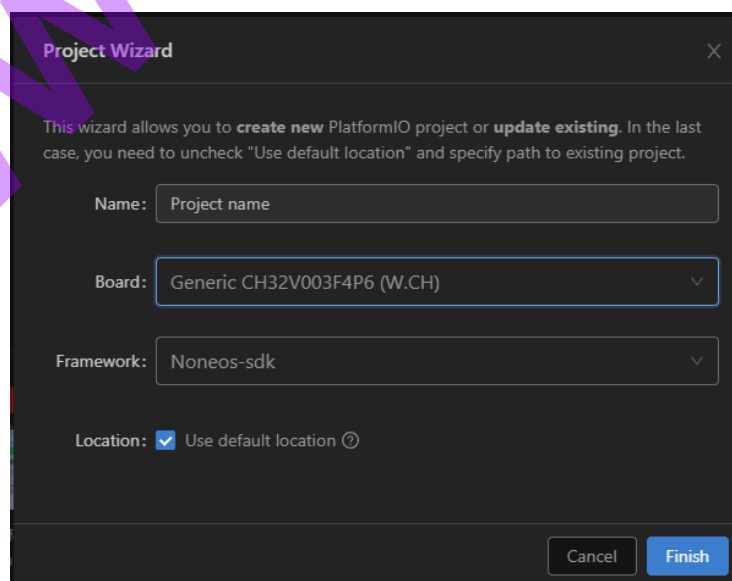
3.2.2. Tạo project

3.2.2.1. Tạo thủ công

Bước 1: Vào giao diện platformIO trên Vscode



Bước 2: Create new project -> New project



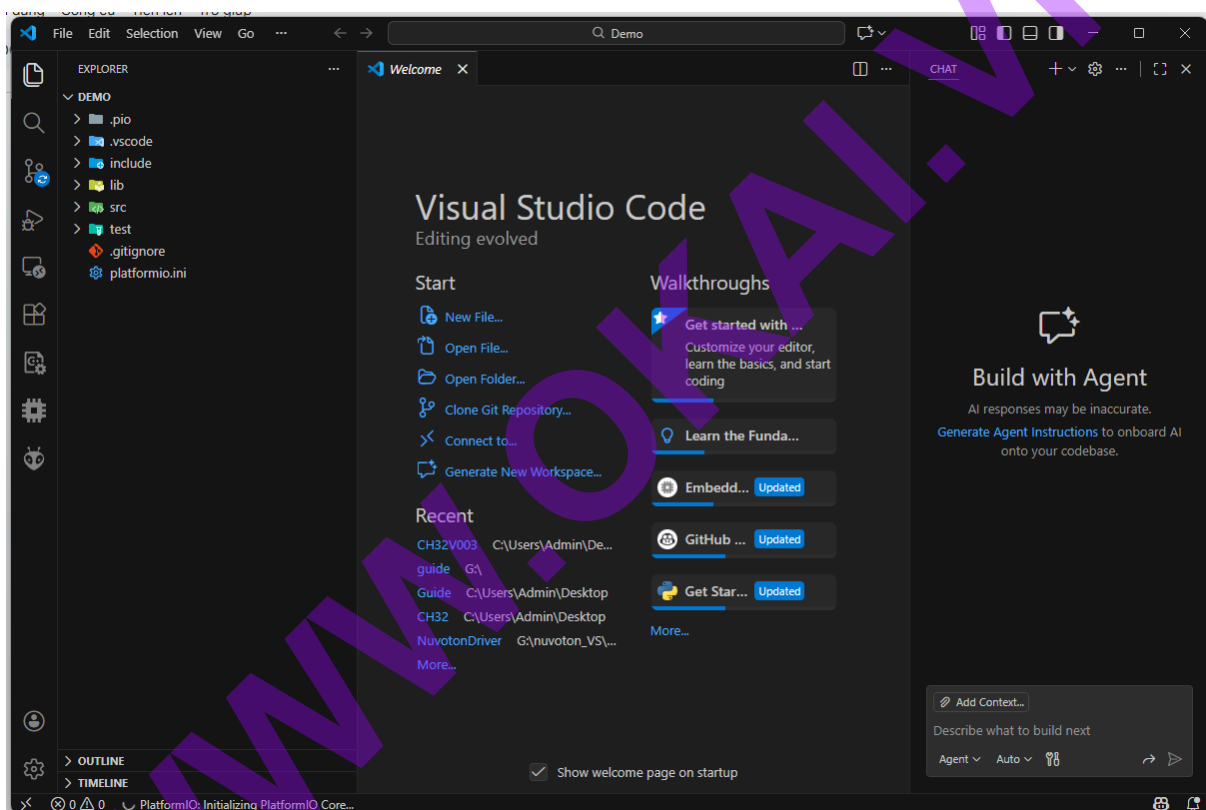
Name: Tên project (tùy ý đặt).

Board: Chọn Generic CH32V003F4P6 (W.CH).

Framework: Noneos-sdk.

Location : Bỏ tick v để chọn nơi lưu dự án.

Sau đó nhấn Finish để xác nhận tạo dự án và sau khi tạo xong sẽ có giao diện như sau:



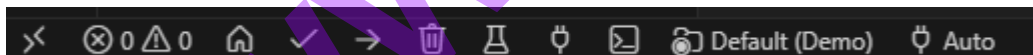
Bước 3: Viết chương trình và nạp code.

Trong thư mục src đã có 1 bản code mẫu bật tắt GPIO sẵn:

```

src > main.c > NMI_Handler(void)
1  #include <ch32v00x.h>
2  #include <debug.h>
3
4  #define BLINKY_GPIO_PORT GPIOC
5  #define BLINKY_GPIO_PIN GPIO_Pin_1
6  #define BLINKY_CLOCK_ENABLE RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE)
7
8  void NMI_Handler(void) __attribute__((interrupt("WCH-Interrupt-fast")));
9  void HardFault_Handler(void) __attribute__((interrupt("WCH-Interrupt-fast")));
10
11 int main(void) {
12     SystemCoreClockUpdate();
13     Delay_Init();
14
15     GPIO_InitTypeDef GPIO_InitStructure = {0};
16     BLINKY_CLOCK_ENABLE;
17     GPIO_InitStructure.GPIO_Pin = BLINKY_GPIO_PIN;
18     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
19     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
20     GPIO_Init(BLINKY_GPIO_PORT, &GPIO_InitStructure);
21
22     uint8_t ledState = 0;
23     while (1) {
24         GPIO_WriteBit(BLINKY_GPIO_PORT, BLINKY_GPIO_PIN, ledState);
25         ledState ^= 1;
26         Delay_Ms(1000);
27     }
28     return 0;
29 }
30
31 void NMI_Handler(void) {}
32 void HardFault_Handler(void)
33 {
34     while (1)
35     {
36     }
37 }

```



Nhìn vào thanh công cụ bên dưới cùng:

Dấu ✓ là để build chương trình.

```

Verbose mode can be enabled via '-v, --verbose' option
CONFIGURATION: https://docs.platformio.org/page/boards/ch32v/genericCH32V003F4P6.html
PLATFORM: WCH CH32V (1.1.0+sha.100e1c1) > Generic CH32V003F4P6
HARDWARE: CH32V003F4P6 48MHz, 2KB RAM, 16KB Flash
DEBUG: Current (wch-link) On-board (wch-link) External (minichlink)
PACKAGES:
- framework-wch-noneos-sdk @ 2.30000.0+sha.f3f1eb7
- tool-openocd-riscv-wch @ 2.1100.240729 (11.0)
- tool-wlink @ 0.23.241116+sha.f44158e
- toolchain-riscv @ 1.120200.220829+sha.d283639
Clock configuration:
Source: hsi+pll
Frequency: 48000000 Hz
Macro: SYSCLK_FREQ 48MHz HSI=48000000
LDF: Library Dependency Finder -> https://bit.ly/configure-pio-ldf
LDF Modes: Finder ~ chain, Compatibility ~ soft
Found 0 compatible libraries
Scanning dependencies...
No dependencies
Building in release mode
Checking size .pio/build/genericCH32V003F4P6/firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM:   [=====]   13.9% (used 284 bytes from 2048 bytes)
Flash: [=====]   8.2% (used 1336 bytes from 16384 bytes)
===== [SUCCESS] Took 0.97 seconds =====

```

Đây là build chương trình thành công

Dấu ☐ là để nạp chương trình.

```

Scanning dependencies...
No dependencies
Building in release mode
Checking size .pio\build\genericCH32V003F4P6\firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [ = ] 13.9% (used 284 bytes from 2048 bytes)
Flash: [ = ] 8.2% (used 1336 bytes from 16384 bytes)
Configuring upload protocol...
AVAILABLE: isp, minichlink, wch-link, wlink
CURRENT: upload_protocol = wch-link
Uploading .pio\build\genericCH32V003F4P6\firmware.elf
Open On-Chip Debugger 0.11.0+dev-02415-gfad123a16-dirty (2024-07-29-15:30)
Licensed under GNU GPL v2
For bug reports, read
http://openocd.org/doc/doxygen/bugs.html
debug_level: 1

Warn : Transport "sdi" was already selected
Ready for Remote Connections
[wch_riscv.cpu.0] Target successfully examined.
** Programming Started **
** Programming Finished **
** Verify Started **
** Verified OK **
** Resetting Target **
shutdown command invoked

===== [SUCCESS] Took 2.00 seconds =====
Terminal will be reused by tasks, press any key to close it.

```

Đây là nạp chương trình thành công

4. Một số lỗi thường gặp

4.1. Không connect được chip

```

Executing task: C:\Users\Admin\.platformio\penv\scripts\platformio.exe run --target upload
Checking size .pio\build\genericCH32V003F4P6\firmware.elf
Advanced Memory Usage is available via "PlatformIO Home > Project Inspect"
RAM: [ = ] 13.9% (used 284 bytes from 2048 bytes)
Flash: [ = ] 8.2% (used 1336 bytes from 16384 bytes)
Configuring upload protocol...
AVAILABLE: isp, minichlink, wch-link, wlink
CURRENT: upload_protocol = wch-link
Uploading .pio\build\genericCH32V003F4P6\firmware.elf
Open On-Chip Debugger 0.11.0+dev-02415-gfad123a16-dirty (2024-07-29-15:30)
Licensed under GNU GPL v2
For bug reports, read
http://openocd.org/doc/doxygen/bugs.html
debug_level: 1

Warn : Transport "sdi" was already selected
Ready for Remote Connections
Error: WCH-Link failed to connect with riscvchip
Error: 1.Make sure the two-line debug interface has been opened. If not, set board to boot mode then use ISP tool to open it
Error: 2.Please check your physical link connection

```

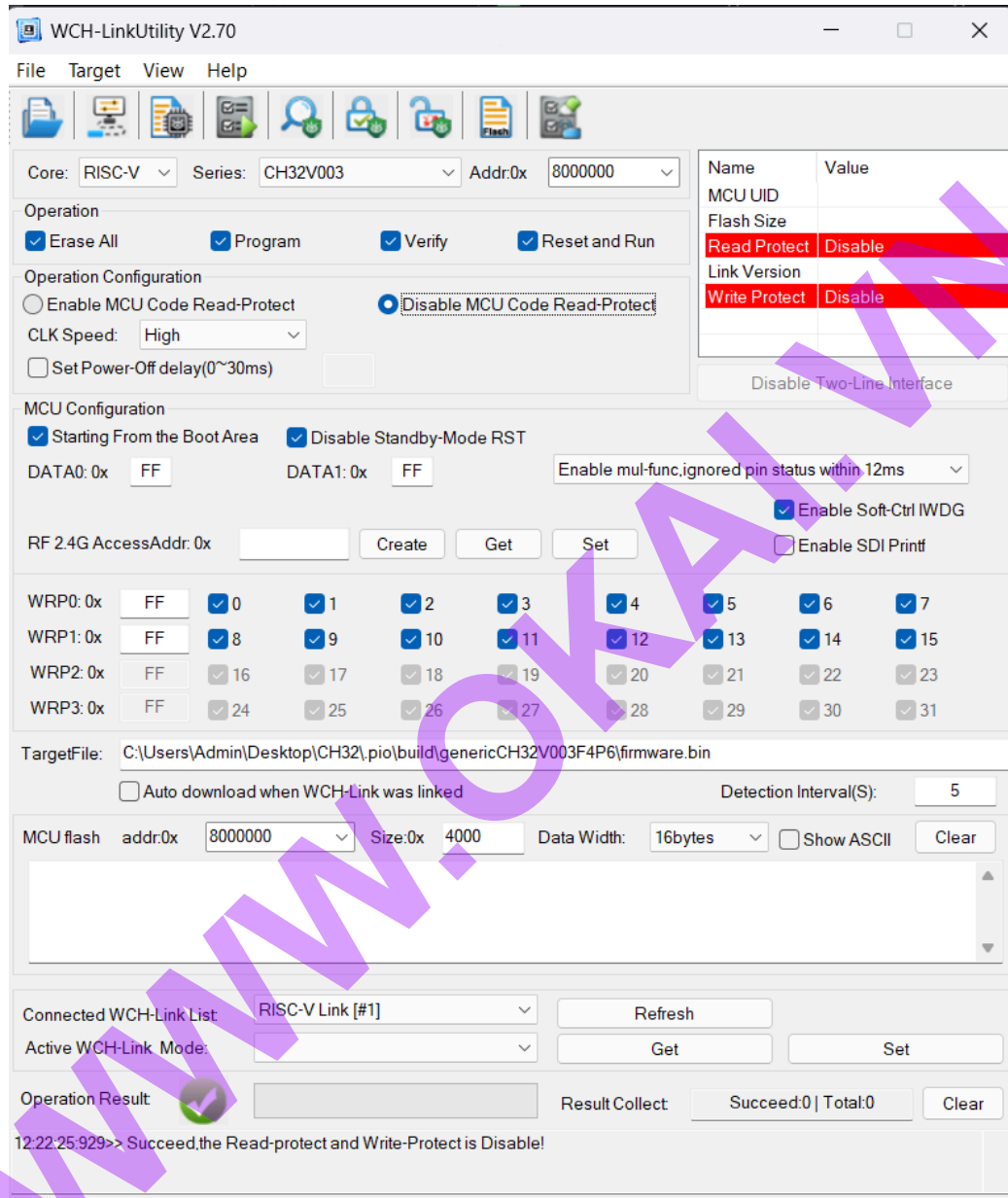
Cách fix: Do tự mò lên có thể không phải tối ưu nhất

Bước 1: Mở phần mềm WCH-LinkUtility

Bước 2: Rút nguồn chip ra

Bước 3 Vừa cắm nguồn lại vừa nhấp vào biểu tượng  trong phần mềm để connect chip

Bước 4 : Nếu mà phần mềm hiển thị như này là thành công và nạp chip như bình thường



★ Lưu ý: Đây là cách tự mò ra lên chưa chắc 100% lần nào cũng thành công